PARSIM: Particles Simulation

José Brás
jose.bras@cern.ch
CERN
Geneva, Switzerland

Pedro Matias
pedro.matias@inesc-id.pt
INESC-ID
Lisbon, Portugal

João Moreira
joao.lobo.moreira@tecnico.ulisboa.pt
Instituto Superior Técnico
Lisbon, Portugal

Abstract

Simulating the interactions of a large number of particles in the *n-body* problem under gravity with collision dynamics is a computationally intensive task. Each simulation time-step involves calculating gravitational forces between many particle pairs, updating particle velocities and positions, and detecting collisions among particles in close proximity. The cost grows quickly with the number of particles, making a straightforward serial implementation very slow for large simulations. To address this, we employ parallelization on distributed-memory multi-core processors using **MPI+OpenMP**. The motivation is to exploit data parallelism inherent in the problem (many particles and spatial regions can be processed independently) to significantly speed up the simulation. Our goal is to optimize the simulation code by restructuring it into parallel phases, minimizing synchronization and communication overhead, and ensuring good load balancing across processes. This report presents our parallelization strategy, the domain decomposition used, key optimizations for performance, synchronization and communication considerations, and an overview of results comparing the optimized distributed version to a serial baseline.

José Brás, Pedro Matias, and João Moreira. . **PARSIM**: Particles Simulation. 4 pages.

1 Introduction

Simulating gravitational and collision dynamics for large numbers of particles in the *n-body* [3] problem is computationally intensive. Each time-step requires computing forces among numerous particle pairs, updating velocities and positions, and detecting collisions. As particle counts grow, naive serial methods become prohibitively slow. To address this, we adopt an MPI+OpenMP [1, 2] approach on distributed-memory multi-core systems, restructuring the simulation into parallel phases while minimizing synchronization and ensuring a balanced workload.

2 Algorithm Overview

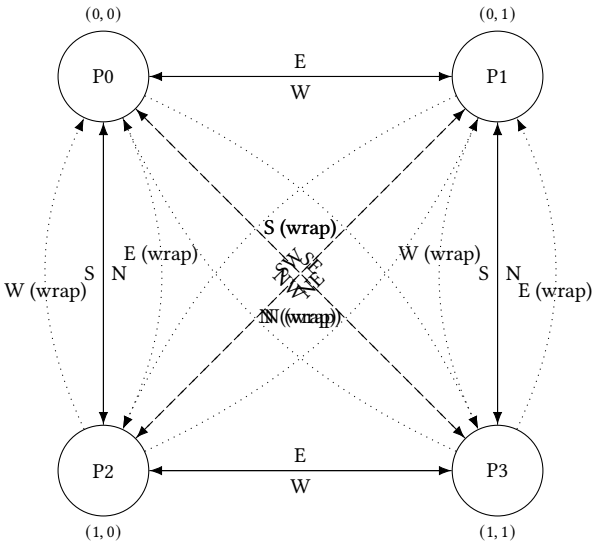
We structure each simulation time-step into a sequence of tasks that operate mostly on independent data, thereby allowing for efficient parallel execution. Specifically, these tasks are: **(1)** resource initialization, **(2)** configuring communication and domain decomposition, **(3)** computing particle centers of mass, **(4)** exchanging center-of-mass data among processes, **(5)** performing gravitational force calculations, **(6)** updating velocities and positions, **(7)** transferring particles that migrate outside local domains, **(8)** recomputing the grid structure, **(9)** detecting and processing collisions, **(10)** retrieving particle 0 from the owning process, and **(11)** gathering collision information on process 0 for final reduction.

Phases **(3)** through **(9)** are repeated t times within the main simulation loop. At program startup, each MPI process spawns its own pool of OpenMP threads, which remain active throughout the run. Barriers at the end of each phase ensure data consistency, while careful partitioning of work across processes and threads supports both load balance and high efficiency.

2.1 Subdomain Decomposition

We employ a checkerboard-style domain decomposition to partition the simulation space among the available MPI processes. Specifically, we invoke `MPI_Cart_create` to establish a two-dimensional Cartesian communicator that assigns each process a coordinate pair (x, y). From these coordinates, every process determines its corresponding subdomain within the global grid, along with its immediate neighbors in all directions (N, NE, E, SE, S, SW, W, NW). This design simplifies boundary handling and promotes efficient data exchange among adjacent processes.

This Cartesian topology is *toroidal*, so processes on one boundary are connected to those on the opposite boundary. Each process thus exchanges data with its up to eight neighbors (including wraparound), populating *ghost regions* that capture external interactions consistently (e.g., gravitational pulls from adjacent subdomains).



The above figure, illustrates the communication layout of the subdomains and shows how each process shares boundary information with its neighbors in a clockwise fashion (from N to NW).

,

Special Cases.

- **Two Processes:** With exactly two processes, the decomposition reduces to a row-wise or column-wise split. Diagonal neighbors effectively collapse into edge neighbors. Because of the toroidal wrapping, certain diagonal references must be remapped to the far boundary of the corresponding dimension.
- **Single Process:** With only one process, it is its own neighbor in every direction due to the toroidal grid, so all boundary exchanges are essentially self-referential.

Finally, although the simulation conceptually uses a grid, we store subdomains in a *flattened* array layout. This approach improves cache locality when accessing particle data, yielding better performance in practice.

2.2 Partitioning

We partition the simulation domain by cells rather than by individual particles to improve cache locality and reduce synchronization overhead. By assigning whole cells to each MPI process and thread, most interactions are handled entirely within one thread, minimizing cross-thread interference. Only when particles move between cells on the same subdomain do we require fine-grained locks.

3 Load Balancing

An even distribution of simulation cells among all MPI processes and OpenMP threads is essential for efficient parallel performance. At the **MPI level**, we compute each process's subdomain boundaries by dividing the global grid dimension `ncside` among the Cartesian grid dimensions `dims`. Any remainder is distributed to the first few processes, ensuring that each process handles as close to an equal number of rows and columns as possible:

```
int rows_per_proc = ncside / dims[0];
int extra_rows    = ncside % dims[0];
int cols_per_proc = ncside / dims[1];
int extra_cols    = ncside % dims[1];

// Compute the starting row index for this process
sub.rowStart = coords[0] * rows_per_proc
  + std::min(coords[0], extra_rows);
sub.rowEnd   = sub.rowStart
  + rows_per_proc + (coords[0] < extra_rows ? 1 : 0);
// Compute the starting column index for this process
sub.colStart = coords[1] * cols_per_proc
  + std::min(coords[1], extra_cols);
sub.colEnd   = sub.colStart + cols_per_proc
  + (coords[1] < extra_cols ? 1 : 0);
sub.localRows = sub.rowEnd - sub.rowStart;
sub.localCols = sub.colEnd - sub.colStart;
}
```

Each process thus obtains a contiguous sub-block of the global domain, eliminating unnecessary overlap and simplifying boundary exchanges.

Once the process subdomain is set, the **OpenMP threads** subdivide their assigned region further. We factor the total number of threads, `n_threads`, into a 2D grid of `T_row` × `T_col`. Each thread locates its (`thread_row`, `thread_col`) position, then splits the process-local rows and columns accordingly, distributing any remainder to the first few threads in each row or column block. If the total thread count exceeds the number of cells, some threads receive no cells (becoming effectively idle), but this approach ensures a straightforward, well-balanced assignment when the problem size is large.

4 Synchronization

We enforce data dependencies between simulation phases with minimal synchronization by placing an `OpenMP` barrier at the end of each major phase (e.g., computing centers of mass, exchanging data, updating positions, and detecting collisions), ensuring that all threads complete their tasks before proceeding. Fine-grained locks are used only when multiple threads update shared cells during particle migration, minimizing race conditions at little overhead. Moreover, we rely on a static decomposition strategy, where each thread pre-computes its own indices and processes its assigned cells directly, eliminating per-iteration scheduling overhead. As a result, barriers only appear at phase boundaries, maintaining correct data dependencies and allowing for improved scalability.

5 Communication

Our simulation uses a two-dimensional Cartesian communicator (`MPI_Cart_create`) to organize processes in a toroidal grid, allowing each process to exchange data only with its up to eight neighbors. This design provides an efficient communication pattern of order $O(P)$, rather than $O(P^2)$. Before the time-steps we first exchange center-of-mass data by issuing a single blocking neighbor-`Alltoall` for counts. Afterwards, done once per timestep we use a non-blocking neighbor-`Alltoallv` for the actual center of mass data, which overlaps with local computations. Notice that the center of mass counts will always be the same. Particle migration follows a similar pattern, but this time, done every time-step: we perform a blocking exchange of particle counts to determine buffer sizes, then launch non-blocking sends/receives to transfer the particles, and integrate arrivals after a final `MPI_Wait`. Lastly, we gather global information such as "particle 0" and collision statistics on process 0. Using non-blocking operations where feasible reduces idle time, while neighbor-based exchanges and minimal synchronization help maintain scalability.

6 Collision Detection

Collision detection is computationally intensive due to the many pairwise comparisons needed to identify colliding particles. In our approach, we use a two-step method to mitigate this cost. First, the sweep-and-prune algorithm sorts particles by their x-coordinate and limits collision checks to those within a narrow x-window, reducing the number of comparisons from $O(n^2)$ to $O(n \log n)$ per subdomain (Algorithm 1). Second, to avoid the drawbacks of recursion (such as deep recursion or stack overflow), we implement an iterative breadth-first search (BFS) to group colliding particles. Since collision chains are typically short, the BFS overhead is minimal. Together, these techniques yield a robust and efficient collision detection process (Algorithm 2).

Algorithm 1: Detect Collisions

```
Input: grid, row_start, row_end, col_start, col_end
for row ← row_start to row_end - 1 do
    for col ← col_start to col_end - 1 do
        cell ← grid[row][col];
        numParticles ← size(cell.particles);
        if numParticles < 2 then
            continue;
        // Define collision constants
        COLLISION_RADIUS ← 5 × 10^−3;
        thresh2 ← COLLISION_RADIUS ×
          COLLISION_RADIUS;
        // Sort particles by x-coordinate
        Sort cell.particles by x-coordinate;
        for i ← 0 to numParticles - 1 do
            pA ← cell.particles[i];
            ax ← pA.x,    ay ← pA.y;
            for j ← i + 1 to numParticles - 1 do
                pB ← cell.particles[j];
                dx ← pB.x − ax;
                if dx > COLLISION_RADIUS then
                    break;
                dy ← | ay− pB.y|;
                if dy > COLLISION_RADIUS then
                    continue;
                dist2 ← dx^2 + dy^2;
                if dist2 ≤ thresh2 then
                    pA.add_colliding(pB);
                    pB.add_colliding(pA);
```

Algorithm 2: Compute collisions

```
Input: Grid, row_start, row_end, col_start, col_end
Output: collision_count, groups
for row ← row_start to row_end - 1 do
    for col ← col_start to col_end - 1 do
        cell ← grid[row][col];
        if size(cell.particles) < 2 then
            continue;
        for p_ix ← size(cell.particles)-1 to 0 do
            par ← cell.particles[p_ix];
            if par.colliding_particles is empty then
                continue;
            cell.remove_particle(p_ix);
            par.reset_mass();
            if par ∈ visited then
                continue;
            group ← empty list;
            Enqueue(par) into queue q;
            Add par to visited;
            while q is not empty do
                current ← Dequeue(q);
                Append current to group;
                foreach neighbor in
                  current.colliding_particles do
                    if neighbor ∉ visited then
                        Add neighbor to visited;
                        Enqueue(neighbor) into q;
            collision_count ← collision_count + 1;
            groups.append(group);
```

7 Performance Evaluation

We evaluated the program using all combinations of 1, 2, 4, 8, 16, 32, and 64 MPI processes, paired with 1, 2, 4, and 8 OpenMP threads. As expected, speedup consistently increased with the number of MPI processes; the more processes employed, the higher the attained acceleration.

Among the tested configurations, using *eight* OpenMP threads offered the best performance. For 64 MPI processes, we were restricted to *4* OpenMP threads by the RNL cluster. We never tested the times or speed ups for instances where the total number of processes is greater than the grid size. Accordingly, every table focuses on the scenario with a maximum number of MPI processes combined with eight threads per process, which proved optimal for a range of particle distributions: high variability or uniform, small or large grids, and both dense and sparse layouts.

The analysis of the speedup results—computed relative to the serial execution times reveals a significant range in parallel efficiency across the tests when using 8 OMP threads per MPI process.

Highest Speedup: For Test 2, the speedup achieved using 32 MPI processes reaches approximately 124.30×. This outstanding performance indicates that for this workload, the hybrid MPI-OMP model scales very effectively. The combination of distributed memory (via MPI) and shared memory (via 8 OMP threads) allows the program to significantly reduce the execution time, suggesting that the workload is highly parallelizable.

Minimum Speedup: In contrast, Test 4 shows a much lower speedup, with MPI 1 achieving only about 3.16× speedup. This relatively modest improvement suggests that the workload in Test 4 has limited parallelism, or that overheads—such as communication and synchronization among the 8 OMP threads—are dominating the performance. It is also possible that the problem size or computational intensity in Test 4 does not scale well with the hybrid approach.

Overall, while the hybrid approach (8 OMP threads per MPI process) can yield substantial speedup for highly scalable problems, the gains vary significantly with the workload characteristics. Tasks with high inherent parallelism and low overhead benefit greatly, whereas workloads with limited parallelism or high synchronization costs exhibit only modest speedup improvements.

Table 1: Execution Times for Serial and MPI Versions featuring 8 OMP Threads

Test	Command	Serial (s)	MPI 1 (s)	MPI 2 (s)	MPI 4 (s)	MPI 8 (s)	MPI 16 (s)	MPI 32 (s)	MPI 64 (s)
1	`1 5000 100 1000000 4`	2.1	0.4	0.2	0.1	0.3	0.0	0.2	0.1
2	`1 5000 100 1000000 100`	49.7	8.5	4.3	16.9	2.0	0.7	0.4	0.8
3	`1 5000 20 1000000 10`	142.6	13.0.1	6.4	2.3	4.0	2.5	N/A	N/A
4	`1 1000 3 10000 10000`	385.6	122.0	80.0	N/A	N/A	N/A	N/A	N/A
5	`3 5000 50 1000000 300`	437.3	61.0	31.1	15.5	16.3	8.9	5.2	N/A
6	`3 5000 50 1000000 500`	727.9	101.6	52.0	26.0	26.8	14.8	8.9	N/A
7	`-1 1000 30 100000 1000`	118.3	21.7	19.3	13.3	25.6	18.1	N/A	N/A
8	`12672 0.05 3 10 10`	0.0	0.0	0.0	N/A	0.0	N/A	N/A	N/A
9	`5893 0.05 3 10 10`	0.0	0.0	0.0	N/A	0.0	N/A	N/A	N/A
10	`8555 0.05 3 10 10`	0.0	0.0	0.0	N/A	0.0	N/A	N/A	N/A
11	`12 100 5 10000 10000`	70.0	20.3	17.1	12.3	N/A	N/A	N/A	N/A
12	`-11 3500 20 500000 10`	132.9	13.8	12.0	10.1	15.9	15.8	N/A	N/A

Table 2: Speedups Compared to Serial Execution Times

Test	Command	MPI 1	MPI 2	MPI 4	MPI 8	MPI 16	MPI 32	MPI 64
1	`1 5000 100 1000000 4`	5.25	10.50	21.00	7.00	N/A	10.50	21.00
2	`1 5000 100 1000000 100`	5.85	11.56	2.94	24.85	71.00	124.30	62.13
3	`1 5000 20 1000000 10`	10.97	22.28	62.00	35.65	57.04	N/A	N/A
4	`1 1000 3 10000 10000`	3.16	4.82	N/A	N/A	N/A	N/A	N/A
5	`3 5000 50 1000000 300`	7.17	14.06	28.21	26.82	49.15	84.10	N/A
6	`3 5000 50 1000000 500`	7.17	14.00	28.00	27.17	49.26	81.87	N/A
7	`-1 1000 30 100000 1000`	5.45	6.14	8.90	4.63	6.54	N/A	N/A
8	`12672 0.05 3 10 10`	N/A	N/A	N/A	N/A	N/A	N/A	N/A
9	`5893 0.05 3 10 10`	N/A	N/A	N/A	N/A	N/A	N/A	N/A
10	`8555 0.05 3 10 10`	N/A	N/A	N/A	N/A	N/A	N/A	N/A
11	`12 100 5 10000 10000`	3.45	4.09	5.69	N/A	N/A	N/A	N/A
12	`-11 3500 20 500000 10`	9.63	11.08	13.16	8.36	8.42	N/A	N/A

8 Conclusion

We have presented an MPI+OpenMP distributed n-body simulation for gravitational and collision dynamics that achieves high speedups while preserving physical accuracy. By structuring the computation into parallel phases, employing a checkerboard domain decomposition with static yet well-balanced partitions, and minimizing synchronization via carefully placed barriers and non-blocking communication, we drastically reduced runtime. We further optimized collision handling by combining a sweep-and-prune broad-phase with a BFS-based method for resolving chained collisions. Future enhancements could include dynamic load balancing for highly non-uniform particle distributions, integration of accelerators (e.g., GPUs) for more efficient force calculations, and refined broad-phase algorithms for improved scalability.

References

[1] Mpi: A message-passing interface standard, version 3.1. https://www.mpi-forum.org/docs/, 2015. Accessed: Mar. 2025.
[2] Openmp application programming interface, version 5.0. https://www.openmp.org/specifications/, 2020. Accessed: Mar. 2025.
[3] Wikipedia contributors. N-body problem. https://en.wikipedia.org/wiki/N-body_problem. Accessed: Mar. 2025.